

lecture_02

August 1, 2026

1 Lecture 2: Problem Types

{note} In the previous lecture, we introduced the five-step process and broadly discussed how optimization problems can be classified. In this lecture, we delve deeper into problem classification - with precise definitions, mathematical formulations, and transportation examples for each type.

Learning Objectives

By the end of this lecture, you will be able to: 1. Classify an optimization problem along three independent axes: functional form, time horizon, and certainty of data. 2. Formulate transportation problems as linear, non-linear, multi-period, and stochastic models with correct notation. 3. Identify modelling assumptions embedded in each problem type and articulate their practical implications.

Prerequisites: Lecture 1 — Five-Step Process; basic algebra and probability (normal distribution, z-scores).

Estimated time: 45–60 minutes.

1.1 Types of Problems

Recall from Lecture 1 that a general optimization problem takes the form:

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{subject to} \quad \mathbf{g}(\mathbf{x}) \geq 0$$

The character of f and $\mathbf{g}$, the time structure of $\mathbf{x}$, and the certainty of the problem's parameters together determine which class a problem belongs to — and therefore which solution methods apply. We organise these dimensions into three independent classification axes:

Axis	Question	Classes
Functional	What is the shape of f and $\mathbf{g}$?	Linear · Non-linear
Temporal	Do decisions and parameters change over time?	Static · Multi-period
Certainty	Are all parameters known with confidence?	Deterministic · Stochastic

These axes are orthogonal — a problem can simultaneously be non-linear, multi-period, and stochastic. Correctly identifying all three dimensions is a prerequisite for selecting an appropriate solution technique.

1.2 1. Functional Classification

The functional classification is determined by whether the objective function and constraints are linear or non-linear in the decision variables.

1.2.1 1.1 Linear Problems

Definition: A linear optimization problem — commonly called a Linear Program (LP) — has both an objective function and all constraints expressed as linear combinations of the decision variables. No products, powers, or non-linear transformations of the variables appear.

When the decision variables are further restricted to integers, the problem is an Integer Linear program (ILP); when some variables are continuous and others integer, it is a Mixed-Integer Linear program (MILP). These are subtypes of LP that require specialised solution techniques.

Example — Freight Vehicle Allocation (ILP)

A textile firm based in Kochi needs to ship 100 tons of goods from Kanchipuram. Two truck types are available for rental: T_1 carries 10 tons at 5,000 per trip, and T_2 carries 20 tons at 8,000 per trip. How many trucks of each type should the firm deploy to minimize total cost?

Let x_1 and x_2 denote the number of T_1 and T_2 trucks deployed, respectively.

Objective:

$$\min_{x_1, x_2} z = 5000x_1 + 8000x_2$$

Subject to:

$$\begin{aligned} 10x_1 + 20x_2 &\geq 100 && \text{(minimum capacity)} \\ x_1, x_2 &\in \mathbb{Z}_+ && \text{(non-negative integers)} \end{aligned}$$

{note} The integrality requirement $x_1, x_2 \in \mathbb{Z}_+$ makes this formally an ILP, not a continuous LP - you cannot deploy half a truck. In general, LP refers to the continuous relaxation; the integer version requires branch-and-bound or cutting-plane methods covered later in the course.

1.2.2 1.2 Non-Linear Problems

Definition: A non-linear optimization problem — called a Non-Linear Program (NLP) — has at least one non-linear term in the objective function or constraints. Non-linearities arise from physical relationships (e.g., quadratic demand curves, exponential decay), economies of scale, or variable interactions.

Non-linear problems are generally harder to solve than LPs. The possible existence of multiple local optima and non-convex feasible regions make NLPs a distinct and richer class requiring specialised methods.

Example — Toll Pricing on the Chennai–Bangalore Expressway (NLP)

A highway management firm wants to set toll prices p_1 (private vehicles) and p_2 (commercial vehicles) to maximize revenue on the Chennai–Bangalore Expressway. Peak-hour demand responds to price:

$$Q_1(p_1) = 5000 - 20p_1, \quad Q_2(p_2) = 6000 - 0.05p_2^2$$

NHAI regulations require: (i) at least 1,000 private and 1,500 commercial vehicles must use the expressway during peak hour, and (ii) toll prices must be at least 150 for each class.

Objective:

$$\max_{p_1, p_2} z = p_1(5000 - 20p_1) + p_2(6000 - 0.05p_2^2)$$

Subject to:

$$\begin{aligned} 5000 - 20p_1 &\geq 1000 && \text{(minimum private flow)} \\ 6000 - 0.05p_2^2 &\geq 1500 && \text{(minimum commercial flow)} \\ p_1 &\geq 150 && \text{(minimum private toll)} \\ p_2 &\geq 150 && \text{(minimum commercial toll)} \\ p_1, p_2 &\in \mathbb{Z}_+ \end{aligned}$$

{note} Expanding the objective yields terms $-20p_1^2$ and $-0.05p_2^3$, confirming non-linearity. The second constraint is also non-linear in p_2 . This prevents the use of simplex. The NLP module will introduce exact methods (penalty method, barrier method, interior-point method) and non-exact methods (local search, evolutionary algorithms, swarm intelligence) to solve such problems.

1.3 2. Temporal Classification

1.3.1 2.1 Static Problems

Definition: A static optimization problem involves decisions made at a single point in time. All parameters are fixed and known, and the solution does not evolve over a time horizon. Both examples in Section 1 are static.

1.3.2 2.2 Dynamic Problems

Definition: A dynamic optimization problem (solved via dynamic programming, Bellman 1957) involves an explicit state variable s_t that (i) summarises all information from past decisions relevant to future decisions, and (ii) transitions deterministically or stochastically according to a known state-transition function $s_{t+1} = T(s_t, x_t)$. The optimal policy is found by decomposing the problem recursively using the Bellman equation:

$$V(s_t) = \min_{x_t \in \mathcal{X}(s_t)} [c(s_t, x_t) + V(T(s_t, x_t))]$$

with boundary condition $V(s_h) = 0$.

The optimal future decision depends only on the current state s_t , not on the full history of how that state was reached.

Example — Pavement Maintenance Scheduling on the NH-44 Corridor (Dynamic Program)

The National Highways Authority of India (NHAI) manages a 300 km stretch of NH-44 (Chennai–Delhi). Each year, it must decide the maintenance action for the corridor. The pavement condition is tracked via the Pavement Condition Index (PCI), an integer score from 0 (failed) to 10 (perfect). Without intervention, the PCI degrades by 1 point per year. NHAI can choose one of three actions:

Action: x_t	Cost (crore/year): $c(x_t)$	PCI change: $\delta(x_t)$
Do nothing (DN)	0	−1
Preventive maintenance (PM)	2	0 (holds current PCI)
Rehabilitation (RH)	8	+3 (restores up to PCI 10)

Additionally, a roughness penalty is incurred by road users when pavement quality is low. The user cost in state s is $u(s) = 10 - s$ crore/year (higher cost when PCI is lower). NHAI must minimize the total cost (agency cost + user cost) over a 5-year horizon, starting from an initial PCI of $s_1 = 7$, with the constraint that PCI must never fall below 4.

State: $s_t \in \{4, 5, 6, 7, 8, 9, 10\}$ — PCI at the start of year t .

Decision: $x_t \in \{\text{DN}, \text{PM}, \text{RH}\}$ — maintenance action in year t .

State transition:

$$s_{t+1} = \begin{cases} \max(0, s_t - 1) & x_t = \text{DN} \\ s_t + 0 & x_t = \text{PM} \\ \max(10, s_t + 3) & x_t = \text{RH} \end{cases}$$

Period cost: $c_t(s_t, x_t) = c(x_t) + u(s_t)$

Bellman equation (backward recursion from $t = 5$ to $t = 1$):

$$V(s_t) = \min_{x \in \mathcal{X}(s)} [c(s_t, x_t) + V(s_{t+1})], \quad V(s_h) = 0 \quad \forall t$$

where $\mathcal{X}(s) = \{x : s_{t+1}(s, x) \geq 4\}$ enforces the minimum PCI constraint.

1.4 3. Certainty Classification

1.4.1 3.1 Deterministic Problems

Definition: A deterministic optimization problem is one in which all parameters are known exactly at the time of decision-making. The objective function value and constraint satisfaction are fully predictable from the input values. All examples above are deterministic.

1.4.2 3.2 Stochastic Problems

Definition: A stochastic optimization problem incorporates uncertainty in one or more parameters, modelled as random variables with known (or estimated) probability distributions. A common approach is chance-constrained programming: a constraint is required to hold with probability at least $1 - \alpha$, where α is the acceptable risk level.

If $\tilde{b} \sim \mathcal{N}(\mu_b, \sigma_b^2)$, the chance constraint $\mathbb{P}(\mathbf{a}^\top \mathbf{x} \geq \tilde{b}) \geq 1 - \alpha$ converts to the deterministic equivalent:

$$\mathbf{a}^\top \mathbf{x} \geq \mu_b + z_{1-\alpha} \cdot \sigma_b$$

where $z_{1-\alpha} = \Phi^{-1}(1 - \alpha)$ is the $(1 - \alpha)$ -th percentile of the standard normal distribution.

Example — Vande Bharat Coach Composition (Stochastic ILP)

Southern Railways plans a daily Vande Bharat service between Chennai and Rameshwaram. Daily passenger demand is uncertain: - Economy class: $D_1 \sim \mathcal{N}(\mu_1 = 900, \sigma_1 = 100)$ - Executive class: $D_2 \sim \mathcal{N}(\mu_2 = 30, \sigma_2 = 10)$

Class	Capacity	Daily op. cost	Revenue per seat
Economy	78 seats	50,000	750
Executive	52 seats	55,000	2,000

Constraints: at least 16 coaches total; demand satisfied on at least 95% of days. Let $x_1, x_2 \in \mathbb{Z}_+$ denote the number of economy and executive coaches, respectively.

Objective (maximize expected daily profit):

$$\max_{x_1, x_2} z = \underbrace{\mu_1 \cdot 750 + \mu_2 \cdot 2000}_{\text{expected revenue}} - \underbrace{(50000x_1 + 55000x_2)}_{\text{operating cost}}$$

Subject to:

$$\begin{aligned}
78x_1 &\geq \mu_1 + z_{0.95} \cdot \sigma_1 && \text{(economy capacity at 95th percentile)} \\
52x_2 &\geq \mu_2 + z_{0.95} \cdot \sigma_2 && \text{(executive capacity at 95th percentile)} \\
x_1 + x_2 &\geq 16 && \text{(minimum coaches)} \\
x_1, x_2 &\in \mathbb{Z}_+
\end{aligned}$$

where $z_{0.95} = \Phi^{-1}(0.95) \approx 1.645$.

{note} Observe that the **objective function** uses fixed expected values μ_1 and μ_2 as constants - it is not itself random. The stochasticity enters exclusively through the **chance constraints**, which are then converted to deterministic equivalents. This conversion is exact only when demand is normally distributed.

Alternative Stochastic Formulations The chance-constrained program in Section 4.2 is one of several ways to handle uncertainty. The choice of formulation encodes a specific assumption about what *solving under uncertainty* means — and different assumptions lead to different decisions and costs. Three additional frameworks are presented below, all applied to the same Vande Bharat coach composition problem.

Framework	Core question	Key assumption	What it ignores
Chance-constrained	Meet demand with probability $\geq 1 - \alpha$	Demand is normally distributed	Cost of unmet demand
Two-stage stochastic (recourse)	Minimize first-stage cost + expected cost of fixing shortfalls	Distribution of demand is known	Worst-case scenarios
Robust optimization	Best solution under the worst-case demand in an uncertainty set	Demand bounded in $[\mu - \Gamma\sigma, \mu + \Gamma\sigma]$	Probabilistic structure
Sample average approximation	Approximate expectation by averaging over sampled scenarios	Scenarios representative of true distribution	Exact distributional form

{note} In the next few lectures, we delve into **Linear Programming** in detail: formulation, the graphical method, the simplex algorithm, sensitivity analysis, and duality - before turning to specialised LP structures (transportation, assignment, trans-shipment, and shortest path problems).

1.5 Further Reading

- Hillier, F.S. & Lieberman, G.J. (2015). *Introduction to Operations Research* (10th ed.), Chapters 1–3. McGraw-Hill.

- Nocedal, J. & Wright, S.J. (2006). *Numerical Optimization* (2nd ed.), Chapter 1. Springer.
- Birge, J.R. & Louveaux, F. (2011). *Introduction to Stochastic Programming* (2nd ed.), Chapter 2. Springer.